

Responda las siguientes preguntas en su documento de entrega:

1. Sobre Observables y Asincronía:

- a. ¿Por qué las peticiones HTTP en Angular devuelven un `Observable` en lugar de la data directa? ¿Qué diferencia hay frente a una `Promise`?
- b. ¿Qué ocurre si un componente llama a un servicio HTTP pero nunca hace `.subscribe()`? ¿Se ejecuta la petición igualmente?

2. Sobre Inyección de Dependencias:

- c. ¿Qué ventaja de modularidad da `providedIn: 'root'` frente a instanciar el servicio manualmente con `new CityService()` dentro de un componente?
- d. ¿Cómo facilitaría la inyección de dependencias la escritura de pruebas unitarias para estos servicios?

3. Sobre Interceptores:

- e. ¿Por qué es mejor centralizar el manejo de errores en `HttpInterceptor` en lugar de poner bloques `try/catch` en cada componente?
- f. Además del manejo de errores, mencione dos casos de uso adicionales para un interceptor.

4. Sobre el Patrón Maestro-Detalle:

- g. ¿Por qué se usa `@Input()` para pasar la ciudad a `CityDetailComponent` en lugar de volver a hacer un GET a `/api/cities/{id}`?
- h. Observe que en la respuesta de GET `/api/cities`, el campo `country` ya viene como objeto anidado (no solo el `countryId`). ¿Qué ventaja de diseño tiene esta decisión del backend para el frontend?

a) Angular devuelve un Observable porque el HTTP es asíncronico y permite componer flujos de uno o varios valores que pueden cancelarse cuando se desuscribe. Un promise emite un solo valor que nunca se cancela.

b) Al nunca hacer un subscribe, no se ejecutan las peticiones. Eso pasa por que los observables solamente arrancan cuando alguien se les subscribe.

c) `providedIn:root` da un servicio singleton global. Es un tree-shackeable descaoplado del componente principal. Si se instancia con un `new CityService()`, se rompe el contenedor y toca resolver cada dependencia manual como `HttpClient`, lo que dificulta el mantenimiento.

d) En las pruebas unitarias, DI facilita inyectar mocks en lugar de implementaciones reales. Esto permite testear servicios y componentes de manera aislada y única.

e) Centralizar errores en un interceptor evita la duplicación y mantiene consistencia y coherencia en los componentes. `try/catch` en componentes no resuelve por sí mismo los errores asíncronos de los observables.

f) Dos usos adicionales pueden ser:

- Adjuntar tokens de autenticacion
- Mantener un logueo de requests y tiempos.

g) Se utiliza `@input` para pasar la ciudad porque el maestro ya tiene esos datos. Evitando así una llamada extra a ciudades de forma redundante.

h) Que country venga anidado en el frontend simplifica el front end, evitando llamadas adicionales para resolver país con cada ciudad, y hace que la tabla detalle sea mas simple de renderizar.